PRACTICAL WORKSHEET 02

ROS2 Publisher, Subscriber & Multi-Node Vision Processing

Module	ROS2 System Development from Scratch
Platform	Ubuntu 24.04 LTS | ROS2 Jazzy | Python 3
Duration	Approximately 3–4 hours (Part A + Part B)
Participant	Name: _____ Group: _____

1. Objective

Develop ROS2 publishers and subscribers, inspect ROS2 topics, publish webcam images, and run independent colour-detection subscribers. Demonstrate that one detector can be modified and restarted while other nodes remain active.

2. Prerequisites / Lab Setup

PW1 completed: ~/mindspark_ws and package mindspark_robot exist. ROS2 Jazzy Desktop, ros-dev-tools and VS Code installed. Laptop webcam available. ESP32 and LEDs are NOT used in PW2.

Task 1 — Set ROS2 environment (every new terminal)

```
source /opt/ros/jazzy/setup.bash
export ROS_DOMAIN_ID=30
export ROS_AUTOMATIC_DISCOVERY_RANGE=SUBNET
source ~/mindspark_ws/install/setup.bash
echo $ROS_DISTRO
echo $ROS_DOMAIN_ID
```

Check / Expected: Outputs should include jazzy and 30. ALL terminals in this lab must use the same ROS_DOMAIN_ID. Domain 30 is the workshop setting, not a universal ROS2 requirement.



Troubleshooting: If a topic/node is missing, check echo $ROS_DOMAIN_ID in BOTH terminals; source the workspace in new terminals. If discovery remains stale, use ros2 topic list --no-daemon or ros2 daemon stop && ros2 daemon start. Keep the publisher running while checking.

3. Part A — Basic Publisher & Subscriber

Task 2 — Create text publisher

In VS Code, create: src/mindspark_robot/mindspark_robot/simple_publisher.py

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimplePublisher(Node):
    def __init__(self):
        super().__init__('simple_publisher')
        self.publisher = self.create_publisher(String, '/message', 10)
        self.count = 0
        self.timer = self.create_timer(1.0, self.publish_message)

    def publish_message(self):
        msg = String()
        msg.data = f'Hello ROS2! Message {self.count}'
        self.publisher.publish(msg)
        self.get_logger().info(f'Publishing: {msg.data}')
        self.count += 1

def main(args=None):
    rclpy.init(args=args)
    node = SimplePublisher()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Task 3 — Create text subscriber

Create: src/mindspark_robot/mindspark_robot/simple_subscriber.py

```
import rclpy
from rclpy.node import Node
from std_msgs.msg import String

class SimpleSubscriber(Node):
    def __init__(self):
        super().__init__('simple_subscriber')
        self.subscription = self.create_subscription(
            String, '/message', self.listener_callback, 10)

    def listener_callback(self, msg):
        self.get_logger().info(f'Received: {msg.data}')

def main(args=None):
    rclpy.init(args=args)
    node = SimpleSubscriber()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Task 4 — Register, build and verify Part A

Open setup.py; add the following lines INSIDE the existing console_scripts list. Retain hello_node from PW1.

```
'simple_publisher = mindspark_robot.simple_publisher:main',
'simple_subscriber = mindspark_robot.simple_subscriber:main',
```

Python

```python
entry_points={
    'console_scripts': [
        'hello_node = mindspark_robot.hello_node:main',
        'simple_publisher = mindspark_robot.simple_publisher:main',
        'simple_subscriber = mindspark_robot.simple_subscriber:main',
    ],
},
```

Build:

```
cd ~/mindspark_ws
colcon build --symlink-install
source install/setup.bash
```

Terminal 1: ros2 run mindspark_robot simple_publisher

Terminal 1 — Publisher

Bash

```
ros2 run mindspark_robot simple_publisher
```

Terminal 2: ros2 run mindspark_robot simple_subscriber

Terminal 2 — Subscriber

Bash

```
source ~/mindspark_ws/install/setup.bash
ros2 run mindspark_robot simple_subscriber
```

Terminal 3: inspect communication while both are running:

```
ros2 topic list
ros2 topic info /message
ros2 topic echo /message --once
ros2 topic hz /message
```

Check / Expected: /message exists; Type: std_msgs/msg/String; Publisher count: 1; Subscription count: 1; messages appear roughly once per second. The first received counter may not be 0.

```
Plain text

Type: std_msgs/msg/String
Publisher count: 1
Subscription count: 1
```

Troubleshooting: If ros2 run cannot find executable: check filename and setup.py entry, rebuild, then source install/setup.bash. If subscription count is 0: launch subscriber and confirm matching topic spelling, types and domain.

4. Part B — Multi-Node Vision Processing

System flow: Laptop webcam → camera_node → /camera/image_raw (sensor_msgs/Image) → red_ball_node & green_ball_node → /red_ball & /green_ball (std_msgs/String).

Task 5 — Install image dependencies and update package

```
sudo apt update
sudo apt install -y python3-opencv python3-numpy ros-jazzy-cv-bridge ros-jazzy-sensor-msgs
```

```
Bash

sudo apt update
sudo apt install -y \
  python3-opencv \
  ros-jazzy-cv-bridge \
  ros-jazzy-sensor-msgs
```

Add to the existing <depend> entries in src/mindspark_robot/**package.xml**:

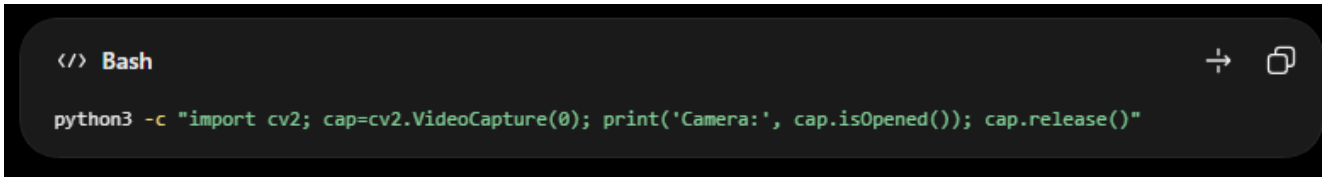
```
<depend>sensor_msgs</depend>
<depend>cv_bridge</depend>
```

```
XML

<depend>rclpy</depend>
<depend>std_msgs</depend>
<depend>geometry_msgs</depend>
<depend>sensor_msgs</depend>
<depend>cv_bridge</depend>
```

Verify laptop webcam availability (close other webcam applications first):

```
python3 -c "import cv2; c=cv2.VideoCapture(0); print('Camera:', c.isOpened()); c.release()"
```



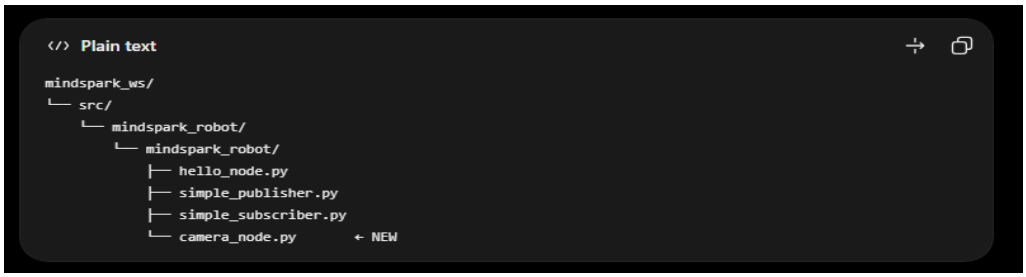
Check / Expected: Camera: True. No image should be requested by multiple independent VideoCapture processes; camera_node is the only node opening the webcam.



Troubleshooting: If Camera: False: close Zoom/browser camera tabs, confirm webcam permission/device index, and test VideoCapture(1) if necessary. GStreamer “Cannot query video position” may be harmless when capture succeeds.

Task 6 — Create camera publisher

Create: src/mindspark_robot/mindspark_robot/camera_node.py




```
import time
import cv2
import rclpy
from rclpy.node import Node
from cv_bridge import CvBridge
from sensor_msgs.msg import Image

class CameraNode(Node):
    def __init__(self):
        super().__init__('camera_node')
        self.publisher = self.create_publisher(Image, '/camera/image_raw', 10)
        self.bridge = CvBridge()
        self.cap = cv2.VideoCapture(0)
        if not self.cap.isOpened():
            raise RuntimeError('Cannot open webcam')
        self.cap.set(cv2.CAP_PROP_FRAME_WIDTH, 320)
        self.cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 240)
        self.frame_count = 0
        self.start_time = time.perf_counter()
        self.timer = self.create_timer(0.1, self.publish_image)
        self.get_logger().info('Camera Publisher Started')

    def publish_image(self):
        ok, frame = self.cap.read()
        if not ok:
            self.get_logger().warning('Failed to capture frame')
            return
        msg = self.bridge.cv2_to_imgmsg(frame, encoding='bgr8')
        msg.header.stamp = self.get_clock().now().to_msg()
        msg.header.frame_id = 'camera'
        self.publisher.publish(msg)
        self.frame_count += 1
        elapsed = time.perf_counter() - self.start_time
        if elapsed >= 5.0:
            self.get_logger().info(
                f'Camera Publishing FPS: {self.frame_count / elapsed:.2f}')
            self.frame_count = 0
            self.start_time = time.perf_counter()

    def destroy_node(self):
        self.cap.release()
        return super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    node = None
    try:
        node = CameraNode()
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        if node is not None:
            node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Register in **setup.py**:

```
'camera_node = mindspark_robot.camera_node:main',
```

Build & Run:

```
cd ~/mindspark_ws && colcon build --symlink-install
source install/setup.bash
ros2 run mindspark_robot camera_node
```

Expected output:

```
[INFO] [camera_node]: Camera Publisher Started
```

NEW terminal (same domain and sourced workspace):

```
ros2 topic list
ros2 topic info /camera/image_raw --verbose
ros2 topic hz /camera/image_raw
```

```
source ~/mindspark_ws/install/setup.bash
ros2 topic list
ros2 topic info /camera/image_raw
ros2 topic hz /camera/image_raw
```

Check / Expected: Topic /camera/image_raw is active, sensor_msgs/msg/Image, 1 publisher, initially 0 subscribers; measured reception approximately 10 Hz at 320×240 on the tested laptop. Observe publishing FPS in camera terminal as well.

Expected:

```
Type: sensor_msgs/msg/Image
Publisher count: 1
Subscription count: 0
```

Publishing rate:

```
average rate: 10.000
```

Troubleshooting: If /camera/image_raw is absent: do not press Ctrl+C on camera node; check domain 30, workspace sourcing and discovery. If image receiving rate is low at 640×480, use 320×240 as shown: BGR8 payload decreases from 921,600 to 230,400 bytes/frame (75% smaller). ros2 topic hz measures received rate; it may differ from the publisher log. On this ROS2 Jazzy CLI, ros2 topic hz does NOT accept --qos-reliability.

Task 7 — Create red ball detection subscriber/publisher

Create: src/mindspark_robot/mindspark_robot/red_ball_node.py

```
import cv2
import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge

class RedBallNode(Node):
    def __init__(self):
        super().__init__('red_ball_node')
        self.bridge = CvBridge()
        self.subscription = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback, 10)
        self.publisher = self.create_publisher(String, '/red_ball', 10)
        self.get_logger().info('Red Ball Detection Started')

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        mask1 = cv2.inRange(hsv, np.array([0, 100, 100]),
                            np.array([10, 255, 255]))
        mask2 = cv2.inRange(hsv, np.array([170, 100, 100]),
                            np.array([179, 255, 255]))
        mask = mask1 + mask2
        contours, _ = cv2.findContours(
            mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        result = String()
        result.data = 'NOT DETECTED'
        if contours:
            largest = max(contours, key=cv2.contourArea)
            if cv2.contourArea(largest) > 200:
                x, y, w, h = cv2.boundingRect(largest)
                cx, cy = x + w // 2, y + h // 2
                result.data = f'DETECTED: x={cx}, y={cy}'
                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 0, 255), 2)
                cv2.circle(frame, (cx, cy), 4, (255, 255, 255), -1)
        self.publisher.publish(result)
        cv2.imshow('Red Ball Detection', frame)
        cv2.waitKey(1)

    def destroy_node(self):
        cv2.destroyAllWindows()
        return super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    node = RedBallNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Add to **setup.py**, build, and launch camera and detector in separate terminals:

```
'red_ball_node = mindspark_robot.red_ball_node:main',
```

```python
'red_ball_node = mindspark_robot.red_ball_node:main',
```

Build package:
cd ~/mindspark_ws && colcon build --symlink-install
source install/setup.bash

```bash
cd ~/mindspark_ws
colcon build --symlink-install
source install/setup.bash
```

Terminal 1: ros2 run mindspark_robot camera_node

Terminal 1 — Camera

```bash
ros2 run mindspark_robot camera_node
```

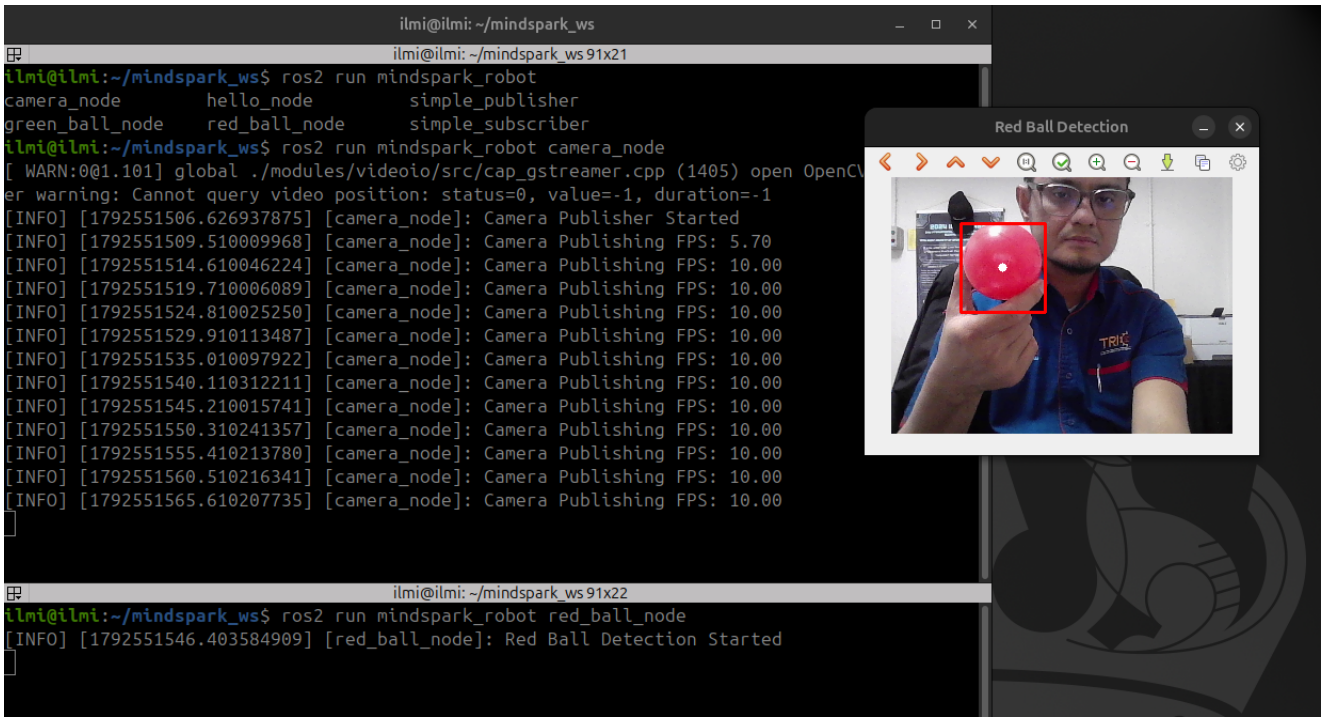
Terminal 2: ros2 run mindspark_robot red_ball_node

Terminal 2 — Red Ball

```bash
ros2 run mindspark_robot red_ball_node
```

Terminal 3: ros2 topic list; ros2 topic echo /red_ball

Check / Expected: Red bounding box appears for red objects; /red_ball sends DETECTED: x=..., y=... or NOT DETECTED. Run ros2 topic info /camera/image_raw: subscriber count should be 1 (plus any CLI subscribers).



Run

ros2 topic list

ros2 topic echo /red_ball



Troubleshooting: If window is blank/no image: confirm camera node running, matching domain and /camera/image_raw active. If no red box: lighting and HSV thresholds affect recognition; avoid red background; adjust area threshold 200 if object is small.

Task 8 — Create independent green ball subscriber/publisher

Create: src/mindspark_robot/mindspark_robot/green_ball_node.py

```
import cv2
import numpy as np
import rclpy
from rclpy.node import Node
from sensor_msgs.msg import Image
from std_msgs.msg import String
from cv_bridge import CvBridge

class GreenBallNode(Node):
    def __init__(self):
        super().__init__('green_ball_node')
        self.bridge = CvBridge()
        self.subscription = self.create_subscription(
            Image, '/camera/image_raw', self.image_callback, 10)
        self.publisher = self.create_publisher(String, '/green_ball', 10)
        self.get_logger().info('Green Ball Detection Started')

    def image_callback(self, msg):
        frame = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')
        hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
        mask = cv2.inRange(hsv, np.array([35, 70, 70]),
                           np.array([85, 255, 255]))
        contours, _ = cv2.findContours(
            mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        result = String()
        result.data = 'NOT DETECTED'
        if contours:
            largest = max(contours, key=cv2.contourArea)
            if cv2.contourArea(largest) > 200:
                x, y, w, h = cv2.boundingRect(largest)
                cx, cy = x + w // 2, y + h // 2
                result.data = f'DETECTED: x={cx}, y={cy}'
                cv2.rectangle(frame, (x, y), (x+w, y+h), (0, 255, 0), 2)
                cv2.circle(frame, (cx, cy), 4, (255, 255, 255), -1)
                cv2.putText(frame, f'GREEN BALL ({cx}, {cy})',
                            (x, max(y-10, 15)), cv2.FONT_HERSHEY_SIMPLEX,
                            0.5, (0, 255, 0), 2)
        self.publisher.publish(result)
        cv2.imshow('Green Ball Detection', frame)
        cv2.waitKey(1)

    def destroy_node(self):
        cv2.destroyAllWindows()
        return super().destroy_node()

def main(args=None):
    rclpy.init(args=args)
    node = GreenBallNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt:
        pass
    finally:
        node.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()
```

Add the following entry to **setup.py** and rebuild while camera and red detector **CONTINUE** running:

```
'green_ball_node = mindspark_robot.green_ball_node:main',
```

Build:

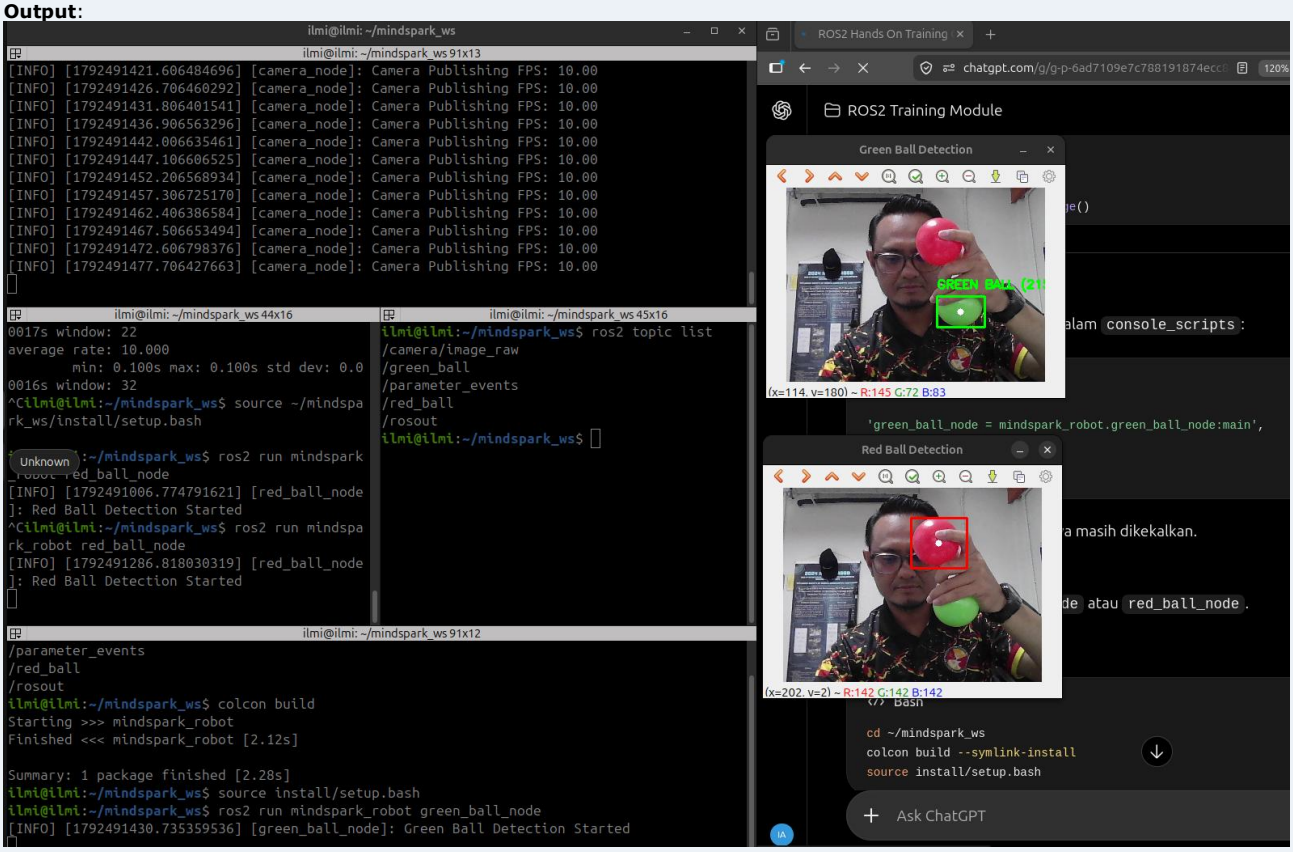
```bash
cd ~/mindspark_ws
colcon build --symlink-install
source install/setup.bash
```

cd ~/mindspark_ws && colcon build --symlink-install
source install/setup.bash

```bash
ros2 run mindspark_robot green_ball_node
```

ros2 run mindspark_robot green_ball_node

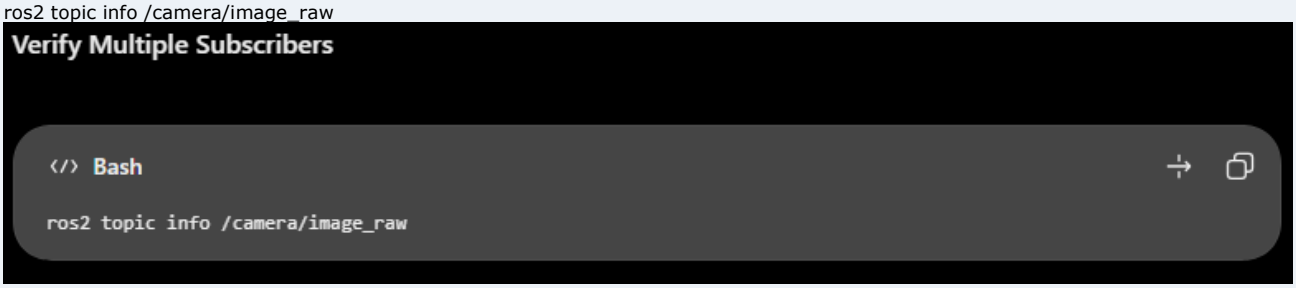
Output:



In a fourth terminal, check active topics and links:

```
ros2 topic list
```

ros2 topic info /camera/image_raw





ros2 topic echo /green_ball --once

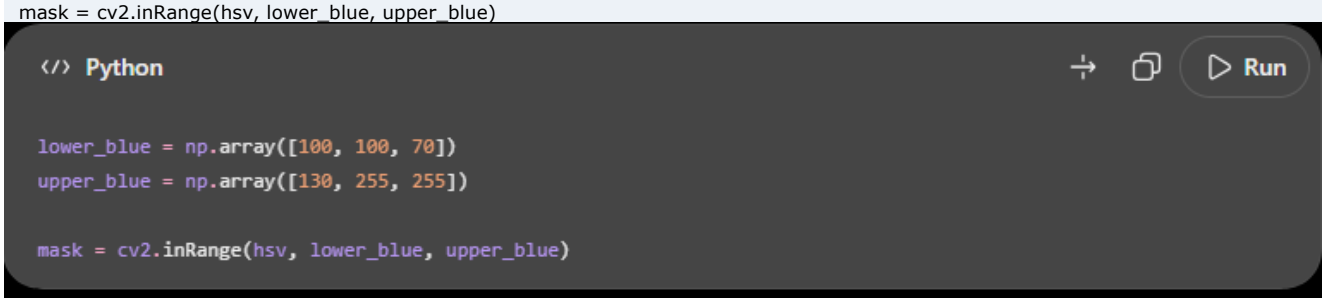
Check / Expected: 1 publisher + 2 subscribers on /camera/image_raw (unless topic hz/echo is also subscribing). Both detection windows stay active; /red_ball and /green_ball topics appear.

Troubleshooting: If two viewers show different frames, they process independent streams and may lag slightly. If detector receives no images, check subscriber count, domain, and camera publisher. Do not open cv2.VideoCapture() inside detectors.

Task 9 — Node independence experiment: green → blue

Stop ONLY green_ball_node (Ctrl+C). Keep camera_node and red_ball_node active. In green_ball_node.py replace the HSV mask with the following:

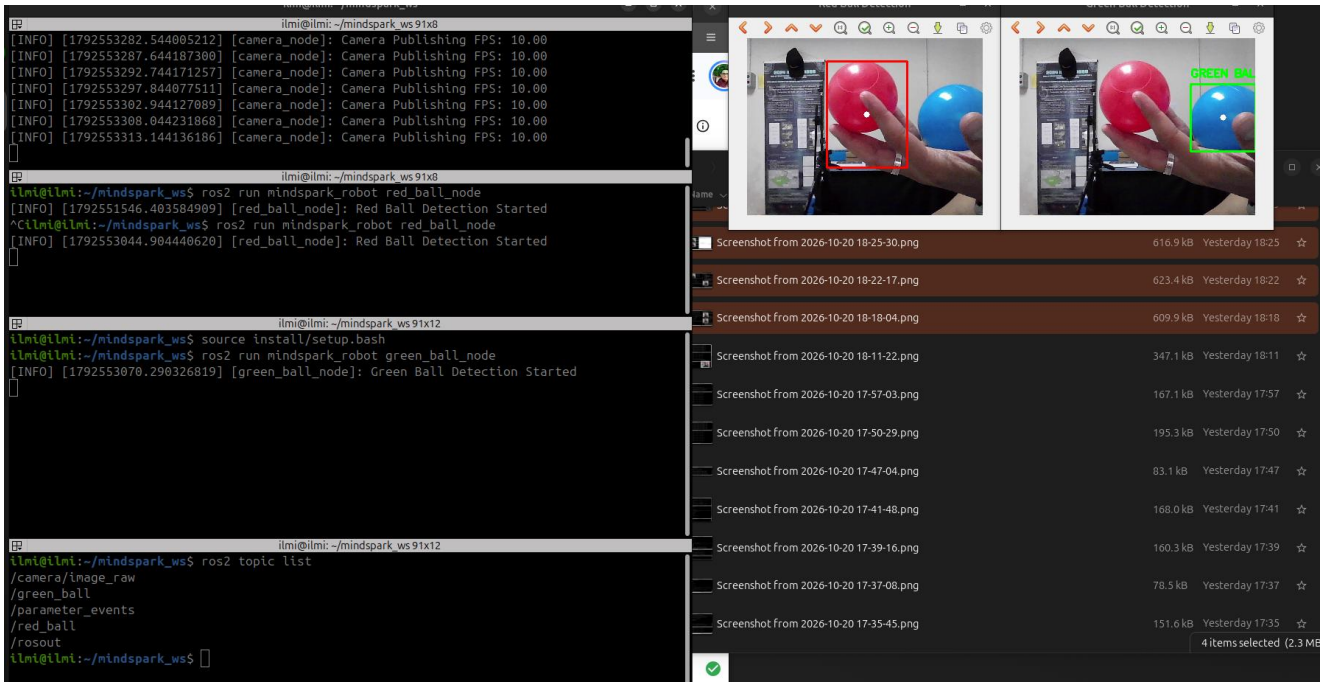
```
lower_blue = np.array([100, 100, 70])
upper_blue = np.array([130, 255, 255])
mask = cv2.inRange(hsv, lower_blue, upper_blue)
```



Also change **box colour** to (255, 0, 0), display **label to BLUE BALL**, and **window title** to Blue Ball Detection. Keep /green_ball and executable name temporarily for the controlled experiment; rename them in a subsequent refactor if desired.

```
ros2 run mindspark_robot green_ball_node
ros2 topic info /camera/image_raw
ros2 topic list
```

Check / Expected: Blue object is detected by restarted node; red detection and camera publish continue uninterrupted. The topic is still named /green_ball unless explicitly changed. Using --symlink-install permits many Python-only edits without rebuilding; rebuild if setup.py/entry points change.



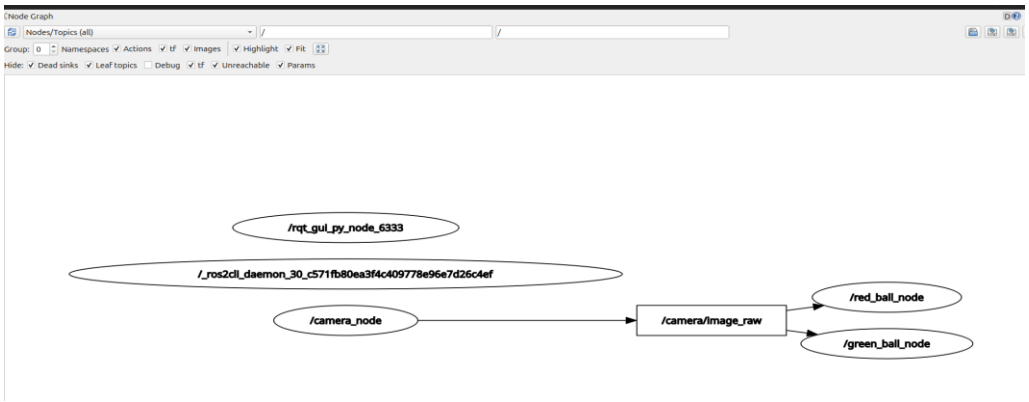
Troubleshooting: If output still says GREEN BALL, replace the text string in cv2.putText. If the old code runs, ensure saved correct source file and fully restart the process; rebuild/source when necessary.

Task 10 — Visualise ROS2 graph

```
rqt_graph
 # If unavailable:
 # sudo apt install ros-jazzy-rqt-graph
```

In rqt_graph, click Refresh. Choose Nodes/Topics (active). If /red_ball or /green_ball are hidden, untick Hide → Dead sinks and Leaf topics; alternatively run ros2 topic echo for each output to make consumers visible.

Check / Expected: Graph shows /camera_node → /camera/image_raw → both detection nodes; colour detections publish to independent topics. Other ROS internal nodes may also appear.



Troubleshooting: If graph is empty or incomplete: confirm all three nodes remain active, same domain, and Refresh. Hide filters may conceal topics with no subscribers.

5. Evidence & Verification

Checkpoint	Pass	Evidence / notes
Part A: /message Publisher=1, Subscriber=1	☐	_____
Camera topic active; Image at 320×240	☐	_____
Camera publishing / receiving ~10 Hz	☐	_____
Red detection + /red_ball output	☐	_____
Green detection + /green_ball output	☐	_____
Image topic has 2 detection subscribers	☐	_____
Green modified to blue; red remains active	☐	_____
rqt_graph shows multi-node architecture	☐	_____

6. Quick Check

1. Why must ROS_DOMAIN_ID match in all workshop terminals? _____
2. What is the difference between publishing FPS and ros2 topic hz? _____
3. Why can two detection nodes use the same /camera/image_raw topic? _____
4. Which changes to green node were made without stopping red node? _____
5. Why was 320×240 selected for the training camera stream? _____